SYSTEM DESIGN

Vector Clocks Visual Guide

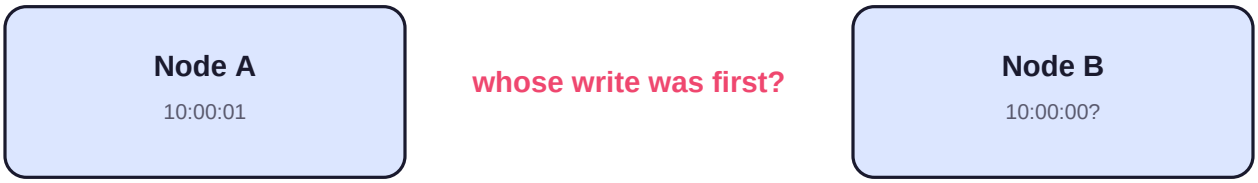
Track causality without a global clock:
happens-before, concurrency, and conflict resolution.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

No Global Clock

Wall clocks skew; NTP lies by milliseconds. Ordering needs something else.



- Two writes 'one millisecond apart' on different machines prove nothing -- clocks disagree by more than that.
- Lamport's insight: track CAUSALITY (did X know about Y?), not wall time.
- Vector clocks: one counter per node -- a version stamp that captures 'who has seen what'.

STEP 2/6

The Vector

Each node keeps [A, B, C]: its own counter plus the latest it has seen from others.

```
A writes:  [1, 0, 0]      B writes:  [0, 1, 0]

A sees B's write, then writes:  [2, 1, 0]
```

- Rule: on a local event, increment YOUR counter. On receiving, take max of each position, then increment yours.
- The vector is a summary of causal history: '[2,1,0]' means 'I have seen 2 of A's events and 1 of B's'.
- No coordinator, no synchronized clocks -- just counters piggybacked on every message.

STEP 3/6

Happens-Before

Compare vectors position by position -- the dominance rule.

```
V1 = [1, 0, 0]   V2 = [2, 1, 0]
```

V1 < V2 on EVERY position => V1 happened-before V2

V2's author had seen V1's event: V2 causally follows V1. Safe to overwrite -- no conflict.

If every counter in V1 is at most V2's, with at least one strictly smaller, V1 precedes V2. The later write wins legitimately.

STEP 4/6

Detecting Concurrent Updates

Neither vector dominates: the writes are concurrent -- a real conflict.

```
V1 = [1, 0, 0]   V2 = [0, 1, 0]
```

V1 not <= V2, V2 not <= V1 => CONCURRENT

- Neither author saw the other's write: both updated from the same ancestor independently.
- The system CANNOT pick a winner -- timestamps would just be guessing.
- Honest answer: keep BOTH versions (siblings) and let the reader reconcile.

STEP 5/6

Dynamo-Style Conflict Resolution

Return siblings; the client merges and writes back the resolution.

- Read returns both versions with their vectors: the app (or user) merges them.
- Merged write carries a vector dominating both ([2,2,0]-style): descendants subsume ancestors.
- Shopping cart: union the items. Last-writer-wins is a policy choice -- vector clocks just expose the truth.

STEP 6/6

Costs and Limits

Vectors grow with the cluster -- and never shrink on their own.

- Size: one counter per node EVER -- 1000 nodes = 1000 counters on every version stamp.
- Pruning: drop entries for dead nodes (risky) or use dotted version vectors (one entry per writer).
- Do not confuse with version NUMBERS: a single integer cannot detect concurrency -- that is the whole point.
- Modern use: DynamoDB, Riak, Cassandra (lightly), and CRDT metadata -- anywhere merges must be honest.

STEP ✓I✓

Key Takeaways

Vector Clocks — Remember

- Wall clocks cannot order distributed events; vector clocks track causality with per-node counters.
- Increment yours locally; merge with max on receive -- the vector summarizes causal history.
- V1 <= V2 everywhere means V1 happened-before V2 (safe overwrite); otherwise the writes are concurrent.
- Concurrent writes become siblings: return both, let the reader merge, write back a dominating vector.
- Vectors grow with cluster size -- prune carefully or use dotted versions.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev